Tutorial: The Chaos Game and Vectorisation

Matteo Fasiolo

The Chaos Game

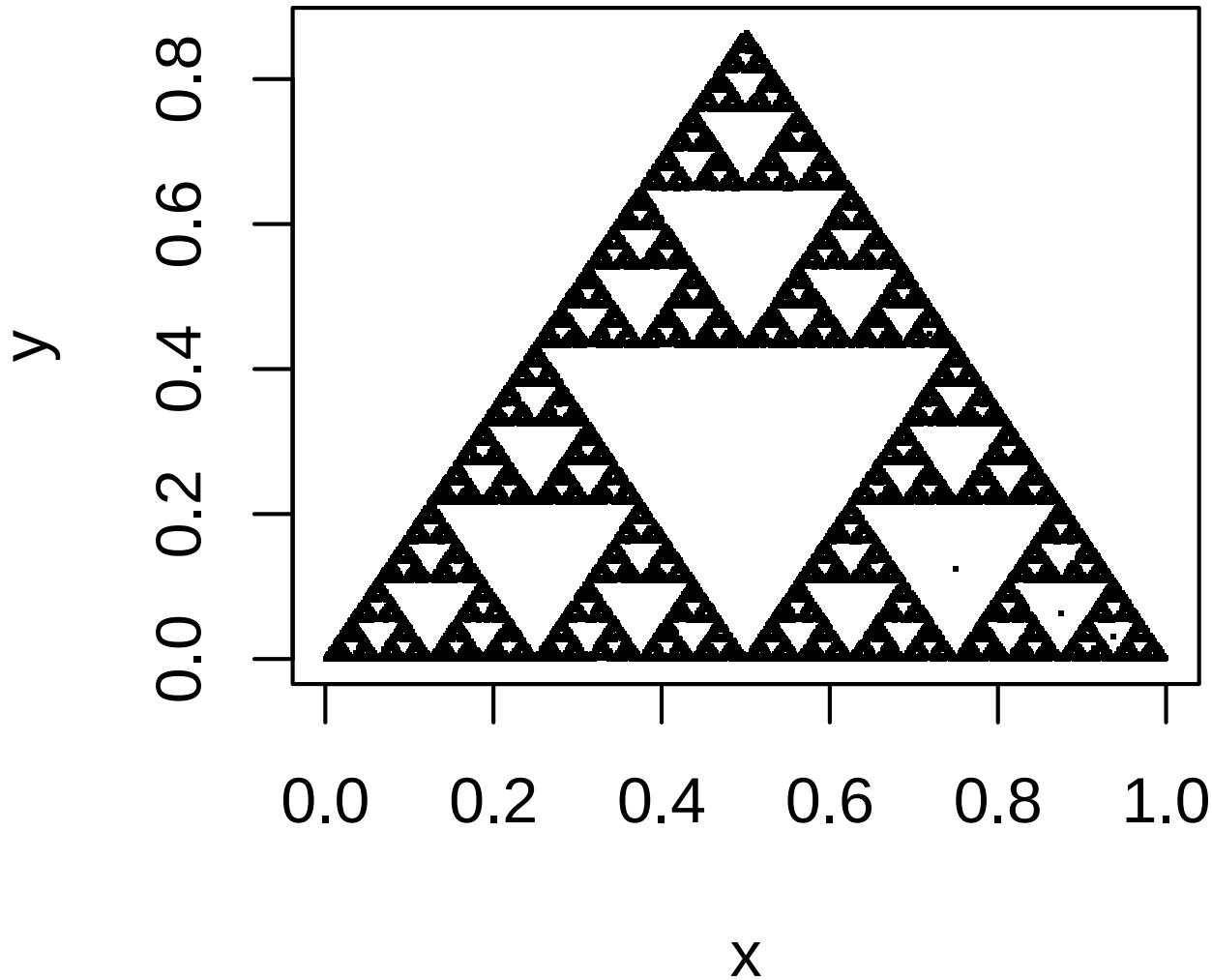
Here is a surprising game that draws a beautiful picture from a very simple rule.

Take the three corners of a triangle. Start somewhere inside it. Then repeat, thousands of times:

1. pick **one of the three corners at random**;
2. move to the **point half-way** between where you are now and that corner;
3. put a dot at your new position.

You would expect a random smudge. Instead you get this...

The target picture



This is the **Sierpinski triangle**. Your job today is to draw it, and then to make your code **fast**.

The plan

The picture is the fun part, but it is the **easy** part.

The real goal of this tutorial is a slow R program **fast** by

- **pre-allocating** storage instead of growing a vector, and
- moving repeated work **out of the loop** and **vectorising** it.

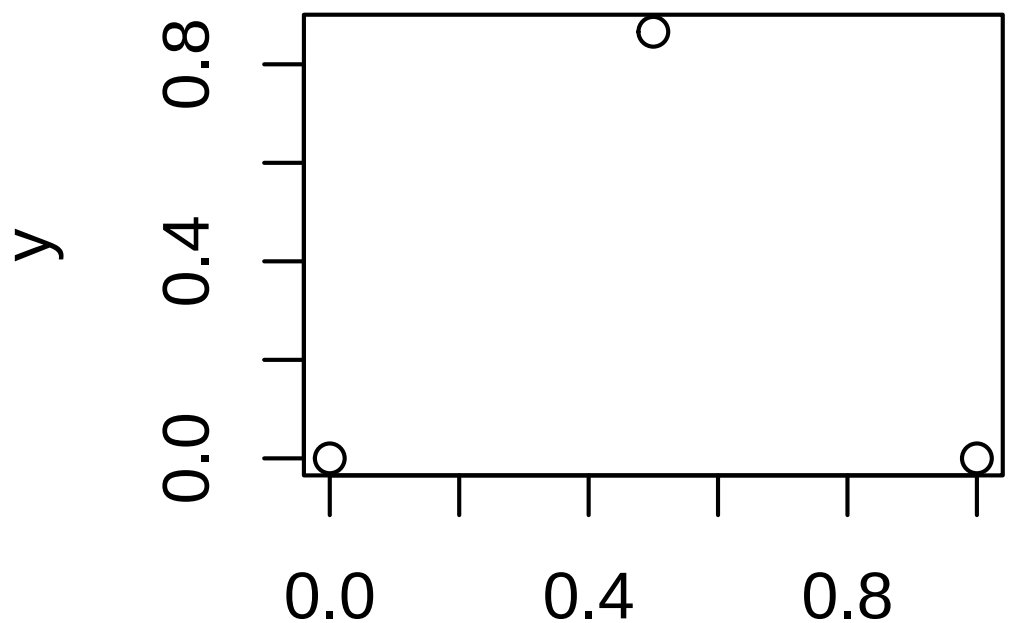
Keep the lecture slide on "Speeding up R" next to you.

Step 1: the corners

We store the three corners as the rows of a 3×2 matrix. Row `k` holds the (x, y) coordinates of corner `k`.

```
V <- matrix(c(0,   0,             # corner 1
              1,   0,             # corner 2
              0.5, sqrt(3)/2),    # corner 3
            nrow = 3, ncol = 2, byrow = TRUE)

plot(V[, 1], V[, 2], xlab = "x", ylab = "y")
```



Step 2: a first (slow) version

Complete the skeleton below. Each step picks a random corner `k`, moves half-way to it, and **stores** the new point. To turn a random `u` in $[0, 1)$ into a corner 1, 2 or 3 we use `as.integer(u * 3) + 1`.

```
chaos_slow <- function(n){
  x <- 0.5; y <- 0.25
  xs <- c(); ys <- c()          # empty results
  for(i in 1:n){
    u <- runif(1)
    k <- as.integer(u * 3) + 1  # corner 1, 2, 3
    # TODO: move half-way to corner k (x, y)
    xs <- c(xs, x)              # grow one element
    ys <- c(ys, y)              # at a time -- slow!
  }
  return(cbind(xs, ys))
}
```

Draw it

Once your function works, draw the result with a single `plot()`:

```
pts <- chaos_slow(3e4)
plot(pts[, 1], pts[, 2], pch = ".")
```

Questions

1. Complete `chaos_slow` and check that you obtain the Sierpinski triangle.
2. Try `pch = "."` versus the default. Which is better for 30 000 points?
3. Add the three corners on top of your picture in red, using `points(V[, 1], V[, 2], col = "red")`.

Step 3: how fast is it?

Time your function for a few values of `n`, using `Sys.time()`:

```
for(n in c(2000, 20000, 100000)){
  tic <- Sys.time()
  chaos_slow(n)
  print( Sys.time() - tic )
}
```

Questions

4. Roughly, when you multiply `n` by 10, does the time multiply by about 10, or by a lot more?
5. A run with `n = 1e5` may take **many seconds**. That is shocking for such a tiny calculation. Where is all the time going?

Why is it slow?

Two things are wrong, and both were warned about in Week 8A.

Problem 1: growing a vector. The line

```
xs <- c(xs, x)
```

does **not** just add one number. Each time, R makes a **brand-new, longer** vector and copies everything across. Over `n` steps that is roughly $1 + 2 + \cdots + n \approx n^2/2$ copies: the cost grows like n^2.

Problem 2: calling `runif(1)` `n` times. Each call has an overhead. Calling `runif` once for a single number, `n` times, wastes most of that effort.

Fix 1: pre-allocate the storage

Instead of growing `xs` and `ys`, make the result matrix **once, at full size**, and fill in row `i` at step `i`.

```
chaos_prealloc <- function(n){
  x <- 0.5; y <- 0.25
  pts <- matrix(0, nrow = n, ncol = 2)  # allocate now

  for(i in 1:n){
    u <- runif(1)
    k <- as.integer(u * 3) + 1

    x <- (x + V[k, 1]) / 2
    y <- (y + V[k, 2]) / 2

    pts[i, ] <- c(x, y)      # fill an existing row
  }                          # (no copying!)

  return(pts)
}
```

Fix 2: vectorise the randomness

We still call `runif(1)` inside the loop `n` times. The same corner formula works on a **whole vector**, so we can make **all** the choices in **one** call, before the loop:

```
choices <- as.integer(runif(6) * 3) + 1   # 6 draws
choices
```

```
[1] 3 2 1 1 1 2
```

We can then look up **all** the chosen corners at once, because a matrix can be indexed by a whole **vector** of row numbers:

```
V[choices, ]   # one row per chosen corner
```

```
     [,1]      [,2]
[1,]  0.5 0.8660254
[2,]  1.0 0.0000000
[3,]  0.0 0.0000000
[4,]  0.0 0.0000000
[5,]  0.0 0.0000000
```

Fix 2: the fast version

```
chaos_fast <- function(n){
  choices <- as.integer(runif(n) * 3) + 1  # all picks
  Vc <- V[choices, ]                       # all corners, no loop

  pts <- matrix(0, n, 2)                   # pre-allocated (Fix 1)
  x <- 0.5; y <- 0.25
  for(i in 1:n){
    # TODO: use Vc[i, 1] and Vc[i, 2]
    #       in place of V[k, 1], V[k, 2]
    pts[i, ] <- c(x, y)
  }
  return(pts)
}
```

Questions

7. Complete `chaos_fast` and check it still draws the triangle.
8. Time all three versions at `n = 1e5`.

What we can, and cannot, vectorise

Notice we **could not** delete the `for` loop entirely.

Each new point depends on the **previous** one:

$$p_i = \tfrac{1}{2}\left(p_{i-1} + \text{corner}\right).$$

This is a **data dependency**: point `i` cannot be computed until point `i-1` is known, so the steps must happen in order.

The lesson is: **vectorise everything you can** (here: the storage and the random draws). Here one genuinely sequential loop has to remain.

Step 4: a fully vectorised measurement

Some questions about the finished picture need **no loop at all**. Suppose `pts` is your matrix of points.

Questions

9. Using a **single line, no loop**, count how many points have `x`-coordinate less than `0.5`.
   - Hint: recall that `sum()` is vectorised.
10. Using a **single line, no loop**, compute the Euclidean distance from **every** point to corner 1, which is at the origin $(0, 0)$.

Challenge

11. **A different ratio.** Replace the factor $1/2$ by a factor `ratio` (so `x <- x + ratio * (V[k,1] - x)`). Try `ratio` of 0.4 and of 0.6. How does the picture change?

12. **Four corners.** Put the four corners of a **square** in `V` and pick one of four at random. Does a square Sierpinski pattern appear? (It may not, experiment!)

13. **Colour by corner.** In `chaos_fast` you already have the vector `choices`. Colour each point by the corner it jumped to:

```
plot(pts[, 1], pts[, 2], pch = ".", col = choices)
```

Summary

- **Try to avoid growing a vector with `c()` inside a loop**: pre-allocate with `matrix()` or `numeric()`.
- **Try to pull repeated work out of the loop** and do it in one vectorised call (here, `runif(n)` and a vectorised matrix lookup).
- Some loops are genuinely **sequential** and cannot be removed, but you can still vectorise everything around them.